Java-Safe C++: A Bytecode Compiler and Managed Virtual Machine for a Memory-Safe C++ Subset

Vaibhav Goel
B.Tech. Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
[email address]

[Name of Member 2]
B.Tech. Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
[email address]

[Name of Member 3]
B.Tech. Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
[email address]

***Abstract*—In C++, reading outside an array, following a null pointer or touching freed memory is undefined behaviour: the program may crash, print garbage or keep running on corrupted data. We describe Java-Safe C++ (JSCPP), a compiler and stack-based virtual machine for a small C++ subset in which these failures cannot go unnoticed. The compiler inserts null checks, bounds checks and reference-counting instructions into the bytecode it emits, and the virtual machine turns every violated check into a named exception with a source line. A short list of language restrictions (no address-of operator, no pointer arithmetic, no pointers stored inside heap objects) keeps these guarantees valid and makes reference cycles impossible. The implementation is 3,638 lines of C++17 with 34 instructions. We audited the first working version by building it and running probe programs, found 20 faults (4 of them critical), and fixed all of them. The final system passes 62 automated tests on Windows and Linux. The 12 example programs that finish normally allocate 24 heap objects and free every one. In our measurements the interpreter is 6 to 32 times slower than optimised native C++ on three small benchmarks, which is the price of checked, interpreted execution.**

***Index Terms*—memory safety, compiler construction, bytecode virtual machine, reference counting, C++ subset**

I. Introduction

Memory-safety bugs remain a leading cause of security vulnerabilities. Microsoft reports that roughly 70% of the vulnerabilities it assigns a CVE each year are memory-safety issues [1], and the problem has been studied for decades [2]. In C++ the language definition leaves the behaviour of an out-of-range access or a null dereference undefined, so the compiler is under no obligation to detect them [3]. Managed runtimes such as the Java Virtual Machine take the opposite approach: every access is checked, and a violation raises an exception [4].

This paper asks how much of that discipline can be given to C++-style source code by a small compiler and virtual machine that we can understand completely. Our answer, Java-Safe C++ (JSCPP), accepts a deliberately small subset of C++ and compiles it to a custom bytecode. The compiler writes the safety checks into the bytecode, so the programmer never has to; the virtual machine executes them and manages memory with reference counting.

The contributions of this work are:

1) a C++ subset chosen so that the safety guarantees hold by construction, together with the reason each excluded feature was excluded (Section III);
2) a complete compiler and virtual machine, written from scratch without external tools (Section IV);
3) an ownership convention that lets generated code maintain exact reference counts across assignments, calls, returns and `delete` (Section V);
4) an evidence-based evaluation: an audit of the first version, an automated test suite, memory-balance measurements and a performance study (Section VI).

We do not claim a new safety technique. The work is an engineering study of how far a small, fully inspectable system can go, and of what it costs.

II. Background and Related Work

Retrofitting safety onto C. CCured infers which pointers are provably safe and adds run-time checks to the rest [5]. Cyclone changes the language itself, adding pointer kinds and region-based memory management so that programs are safe by design [6]. Both aim at real C code bases. JSCPP makes the opposite trade: it supports far less, so that no inference is needed and every guarantee can be explained in a sentence.

Safe systems languages. Rust enforces memory safety statically through ownership and borrowing, without a garbage collector [7]. JSCPP also uses an ownership idea, but only inside the code generator, to keep reference counts exact; the programmer sees ordinary C++ syntax.

Detecting errors in unmodified programs. AddressSanitizer instruments compiled code and reports out-of-bounds accesses and use-after-free at run time [8]; Valgrind does similar checking by rewriting binaries while they run [9]. These tools find bugs but leave the language unsafe. In JSCPP

the checks are part of the language implementation and cannot be switched off.

Managed runtimes and reference counting. The Java Virtual Machine defines a stack-based bytecode with checked array and reference operations [4]; JSCPP follows that structure on a much smaller scale. Reference counting for automatic reclamation dates back to Collins [10]. Its well-known weakness is that cycles are never freed; Section V-C shows why the JSCPP subset cannot create them.

Method. The phase structure follows standard compiler texts [11], [12].

III. The Language Subset

A JSCPP program is a set of functions, one of which is `main`. The types are `int` (32-bit), `bool`, `void` (as a return type only), pointers to `int` or `bool`, and local fixed-size arrays of `int` or `bool`. Statements are declarations, `if`/`else`, `while`, `for`, `return`, `delete` and blocks. Expressions include arithmetic, comparison, short-circuit `&&` and `||`, assignment, compound assignment, `++`/`--`, `new`, dereference and indexing. Functions may be recursive and may be called before they are defined. The built-in `print` writes an `int`. Conditions must have type `bool`, and `int`, `bool` and pointers never convert into each other.

Each excluded feature would break at least one guarantee, and each is rejected with its own error message:

- *Address-of (`&`), pointer arithmetic, casts.* They let a program build a pointer that did not come from `new`, so no check on the pointer could be trusted.
- *Pointers to pointers and arrays of pointers.* A heap cell could then hold a reference, which allows reference cycles (Section V-C).
- *Global variables.* The virtual machine only has per-call local slots; a global would silently read the wrong slot.
- *A non-void function that can end without `return`.* Its value would be undefined in C++, so the compiler checks that every path returns.
- *Classes, templates, the standard library, the preprocessor.* Out of scope; the lexer rejects them early with an explanatory message.

IV. System Design

Figure 1 shows the pipeline. Table I gives the size of each part; the whole system is 3,638 lines of C++17 and uses only the standard library.

TABLE I
SOURCE SIZE BY COMPONENT (LINES OF CODE).

Component	Lines
Lexer	369
Parser and AST	982
Semantic analysis	657
Bytecode and code generator	851
Virtual machine and heap	563
Common and command line	216
Total	3,638

A. Front end

The lexer is a hand-written scanner that records a line and column for every token, which later lets errors quote the source line with a caret. It also rejects values that do not fit in a 32-bit `int`. The parser is a recursive-descent parser with one function per precedence level. It rewrites `for` loops into `while` loops, and `++`, `--` and compound assignment into plain assignments, but only when the target has no side effects (a variable, `a[i]` or `*p`), so that nothing is evaluated twice.

B. Semantic analysis

Semantic analysis visits the syntax tree twice. The first pass records the signature of every function, which is what allows calls before definitions and recursion. The second pass checks each body: types must match exactly, the left side of an assignment must be a storage location, whole arrays cannot be assigned, and every path of a non-void function must return. The pass stores the type of every expression and declaration for the code generator.

C. Code generation

The code generator assigns each variable of a function its own frame slot, with parameters in the first slots. It compiles `&&` and `||` to conditional jumps so the right operand runs only when needed, and it records the source line of every instruction. Safety instructions are inserted as described in Section V.

D. Virtual machine and bytecode

The machine has an operand stack, a call stack of frames, one array of locals shared by all frames, and a heap that maps integer handles to objects holding a reference count, a size and the cells. Handles are never reused, so a stale handle can never name a newer object. The instruction set has 34 instructions. A call instruction names its target by index in a function table. The `.jbc` file stores every integer as four little-endian bytes, so the same file runs unchanged on every platform, and a verifier checks every opcode, constant index, function index, jump target and allocation size when a file is loaded.

V. Safety Mechanisms

A. Pointers cannot be forged

A pointer value can only come from `new`, from `nullptr`, or by copying another pointer. Because the language has no address-of operator, pointer arithmetic or casts, every non-null pointer names a live (or once-live) heap object. This is the property that makes the run-time checks below sufficient.

B. Null and bounds checks

Every dereference `*p` compiles to `CHECK_NULL` followed by the load or store, and every `a[i]` compiles to `CHECK_BOUNDS` followed by the access. For the statement `*p = a[1];` the compiler emits:

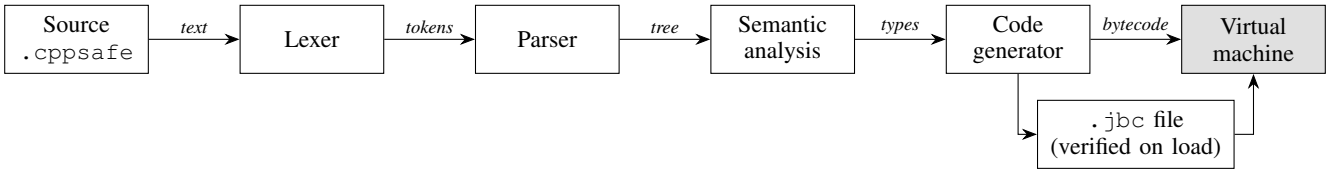


Fig. 1. Pipeline of the JSCPP compiler. Bytecode can be run directly or saved to a `.jbc` file, which the loader verifies before the virtual machine runs it.

```
LOAD_LOCAL  0       ; array a
PUSH_CONST  (=1)    ; index
CHECK_BOUNDS        ; 0 <= index < size
LOAD_ARRAY
LOAD_LOCAL  1       ; pointer p
CHECK_NULL          ; pointer is not null
STORE_POINTER
```

The access instructions repeat the range test themselves, so hand-written bytecode cannot skip the check. A failed check stops the machine with a named exception, the source line and the function name, and JSCPP exits with status 1.

C. Automatic reference counting

Every heap object carries a reference count. The code generator follows one convention:

- Each local pointer or array variable owns one reference.
- `new` and a call that returns a pointer produce an *owned* value (its count already includes the reference being handed over); storing it transfers that reference.
- Reading a variable produces a *borrowed* value; storing it emits `INC_REF`.
- Pointer arguments are passed as owned references, and the callee releases its parameters when it returns.
- `DEC_REF` is emitted when a variable is reassigned (after the new value is retained, so `p = p` is safe), at the end of every block, and on every `return`.
- A pointer result that nobody stores, such as `make();`, is released immediately.

When a count reaches zero the object is freed at once. `delete p` releases `p`'s reference and then sets `p` to `nullptr`; other variables that refer to the same object keep it alive, so `delete` cannot produce a dangling pointer.

Why cycles cannot occur. A cycle needs one heap object to hold a reference to another. Heap cells hold only `int` or `bool` values, because pointers to pointers and arrays of pointers are rejected by the type rules. References therefore live only in frame slots, and the reference graph between heap objects is empty. The usual weakness of reference counting does not apply to this subset. This argument rests on the type rules; we test it but do not prove it formally.

D. Defined arithmetic and bounded resources

Signed overflow is undefined in C++, so the machine computes `+`, `-` and `*` on unsigned 32-bit values and converts back; the result wraps as in Java. `INT_MIN / -1` is handled explicitly and division by zero raises an exception. The call stack is limited to 1000 frames, the operand stack to 100,000 values and the heap to one million live objects, so runaway programs stop with an exception rather than exhausting the host.

VI. Evaluation

All numbers in this section were produced by a script (`collect_results.py`) that runs the real compiler and writes JSON files; the text and tables read those files, so no figure was typed by hand. Measurements were made on a laptop with an 13th Gen Intel Core i7-13650HX processor running Windows 11, compiled with g++ 15.2.0; Linux results use g++ 15.2.0 under WSL. Claims below are *observed* unless stated otherwise.

A. Audit of the first version

Before extending the system we built its first working version and ran small probe programs against it. We found 20 faults: 4 critical (the feature did not work), 8 high (invalid programs accepted or undefined behaviour) and 8 medium (robustness and usability). The critical ones were: (i) `&&` and `||` emitted a jump whose target was never filled in and stayed at address zero, so `if (true || false)` never terminated; (ii) parameters were numbered after the locals of earlier functions, so a call stored its arguments in the wrong slots and a program that should print 42 printed 0; (iii) locals declared at function level were never released, so reference counting freed nothing for them; and (iv) arrays were never released either. All faults were fixed, and each is covered by a regression test.

B. Test suite

The suite has 62 tests. Each of the 17 example programs is run twice, once from source and once after saving to and loading from a `.jbc` file (17 + 17 tests), and the output, exit status and, for the failing demonstrations, the exception text must match exactly. A further 28 invalid programs must each be rejected with a specific message (28 tests). All 62 tests pass on Windows, and all 62 pass on Linux. We also confirmed that the harness reports a failure when an expected output is deliberately changed.

C. Safety outcomes

Table II lists 9 small programs, each performing one unsafe action. 6 are stopped at run time with an exception, one completes with a defined result, and 2 are rejected before running.

TABLE II
RESULT OF EACH UNSAFE ACTION (OBSERVED). "LINE" IS THE SOURCE LINE REPORTED.

Unsafe action	Stage	Result	Line
Store one past the end of an array	run	`ArrayIndexOutOfBoundsException`	7
Read an array at index -1	run	`ArrayIndexOutOfBoundsException`	5
Dereference `nullptr`	run	`NullPointerException`	4
Use a pointer after `delete`	run	`NullPointerException`	5
Divide by zero	run	`DivisionByZeroException`	2
Recurse without a base case	run	`StackOverflowException`	2
Add one to the largest `int`	run	completes, prints -2147483648	–
Take an address with `&`	compile	rejected: compile error	3
Non-void function that can end without `return`	compile	rejected: compile error	1

TABLE III
WALL-CLOCK TIME IN MILLISECONDS (MEDIAN OF 7 RUNS, INCLUDING PROCESS START-UP). RATIO IS VM (RELEASE) DIVIDED BY NATIVE.

Benchmark	Native	VM (Rel.)	VM (def.)	Ratio
`fib(27)`	5	29	198	6×
loop, 3×10^6 iter.	6	197	1526	32×
bubble sort, 1500 items	10	165	1421	17×

D. Memory behaviour

Of the 17 example programs, 12 finish normally. Together they allocate 24 heap objects and free 24, leaving 0 alive at exit. Programs that stop on an exception can leave objects alive at that moment; the machine reclaims them when it exits. A larger program, `showcase.cppsafe` (161 lines, 11 functions, using recursion, sorting, binary search, pointers, `delete` and overflow), produces 979 tokens, a tree of 704 nodes and 732 instructions. Of these, 14 are `CHECK_NULL`, 14 are `CHECK_BOUNDS`, 7 are `INC_REF` and 21 are `DEC_REF`, all inserted by the compiler. The machine executes 230,016 instructions, allocates 5 objects and frees 5. The program's 28 printed values were also compared with an independent Python implementation of the same computation (`verify_showcase.py`), and all matched.

E. Performance

We compared the virtual machine with the same algorithm written in C++ and compiled with `g++ -O2`. "VM (Rel.)" is the JSCPP binary built in Release mode (`-O3`); "VM (def.)" is the default build without optimisation flags. All three benchmarks print identical results in the virtual machine and in native code. Table III shows that the Release interpreter is 6 to 32 times slower than native code. Process start-up (9 ms for the virtual machine, 4 ms for the native program) is included in every time, so the true ratio for the shortest benchmark is higher than shown. Building JSCPP itself with optimisation makes it 7 to 9 times faster than the default build. We did not compare with the Java Virtual Machine; it was not available on the test machine.

VII. LIMITATIONS AND THREATS TO VALIDITY

- *Evidence, not proof.* The guarantees follow from the type rules and the checks, but we have not proved them formally. Our evidence is a hand-written suite, not a fuzzing or differential-testing campaign.
- *Small language.* There are no structs, strings, global variables or heap arrays, and `print` accepts only an `int`. The guarantees apply to this subset only.
- *Exceptions end the program.* The language has no `try`/`catch`; a failed check stops the program, as an uncaught exception does in Java.
- *Cost of safety.* The checks are never removed, and the interpreter has no just-in-time compiler. The benchmarks are small, run on one machine, and cannot separate the cost of the checks from the cost of interpretation.
- *Audit scope.* The 20 faults were found by us on our own first version; other faults may remain.

Natural next steps are differential fuzzing against native C++, structs with reference-counted fields and a cycle collector, run-time-sized heap arrays, and removing checks that the compiler can prove unnecessary.

VIII. CONCLUSION

JSCPP shows that a compiler and virtual machine small enough to read in an afternoon can turn the most common C++ memory errors into compile-time rejections or named run-time exceptions, and can reclaim memory exactly with reference counting. The price is a restricted language and execution that is many times slower than native code. The audit of the first version also showed how easily such a system can appear to work while its main features fail, which is why the final system is backed by an automated test suite. The source code, tests and demonstration programs are available at https://github.com/Vaibhav8075/java-safe-cpp.

REFERENCES

[1] MSRC Team, "A proactive approach to more secure code," Microsoft Security Response Center blog, Jul. 2019, accessed: Oct. 28, 2026. [Online]. Available: https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code.

[2] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal war in memory," in *Proc. IEEE Symp. Security and Privacy*, 2013, pp. 48–62.

[3] ISO/IEC, "ISO/IEC 14882:2017, programming languages — C++," International Organization for Standardization, Geneva, Switzerland, Tech. Rep., 2017.

[4] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, *The Java Virtual Machine Specification, Java SE 17 Edition*. Oracle America, Inc., 2021.

[5] G. C. Necula, S. McPeak, and W. Weimer, "CCured: Type-safe retrofitting of legacy code," in *Proc. 29th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, 2002, pp. 128–139.

[6] T. Jim, J. G. Morrisett, D. Grossman, M. W. Hicks, J. Cheney, and Y. Wang, "Cyclone: A safe dialect of C," in *Proc. USENIX Annual Technical Conf.*, 2002, pp. 275–288.

[7] N. D. Matsakis and F. S. Klock, "The Rust language," in *Proc. 2014 ACM SIGAda Annual Conf. High Integrity Language Technology (HILT)*, 2014, pp. 103–104.

[8] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "Address-Sanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conf.*, 2012, pp. 309–318.

[9] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. 28th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100.

[10] G. E. Collins, "A method for overlapping and erasure of lists," *Commun. ACM*, vol. 3, no. 12, pp. 655–657, 1960.

[11] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson Addison-Wesley, 2006.

[12] R. Nystrom, "Crafting interpreters," [Online]. Available: https://craftinginterpreters.com, 2021.